

Assignment 2 — Report

Course: "Molnapplikationer fördjupning"

Student: Claes Fransson **Date:** 2026-06-07

Repository: https://github.com/Claes1981/moln_fordj_inlamningsuppgifter

1. Part 1 — Observability (Structured Logging & Correlation ID)

1.1. Implementation

CorrelationIdMiddleware (`src/CloudSoft.Web/Middleware/CorrelationIdMiddleware.cs`):

- Generates a unique `X-Correlation-ID` (GUID) for every incoming request.
- Reuses an existing correlation ID if the client already sends one in the `X-Correlation-ID` header.
- Adds the correlation ID to the logging scope so that all log messages automatically carry it.
- Placed early in the middleware pipeline (after exception handling) for maximum coverage.

Structured Logging (message templates + `ILogger<T>`):

- `AccountController` and `JobPostingsController` upgraded from `Console.WriteLine` to `ILogger<T>` with message templates.
- Example: `_logger.LogInformation("User logged in: {Username}", model.Username);`
- Log levels chosen deliberately: `Information` for normal operations, `Warning` for suspicious patterns, `Error` for failures.

What is intentionally NOT logged:

- Passwords and authentication tokens are never written to logs.
- Personal data beyond the username is excluded (no email addresses, IP addresses, or file contents).
- API keys and connection strings are never logged.

JSON Console Logging (`appsettings.json`):

- `IncludeScopes: true` ensures the correlation ID and other scope variables accompany every log entry.
- Default level: `Warning` for the root category, `Information` for `CloudSoft` and `Microsoft.AspNetCore`.

- JSON format produces structured output that Azure Monitor can parse automatically.

How logs reach Log Analytics:

The Container Apps Environment is configured in `infra/main.bicep` to forward container stdout/stderr to Azure Monitor:

```
resource caEnvironment 'Microsoft.App/managedEnvironments@2024-03-01' = {  
  name: '${appName}-env-${uniqueSuffix}'  
  location: location  
  properties: {  
    appLogsConfiguration: {  
      destination: 'azure-monitor'  
    }  
  }  
}
```

Because ASP.NET Core's JSON console logger writes to stdout, every structured log message flows through the Container Apps log pipeline directly into Log Analytics. No additional SDK or agent is required.

KQL query for incident investigation:

To find all failed login attempts with their correlation IDs:

```
AppServiceConsoleLogs  
| where Log contains '"EventId"'  
| where Log contains 'Warning' or Log contains 'Error'  
| project TimeGenerated, Log, ContainerGroupName  
| order by TimeGenerated desc  
| take 50
```

This query retrieves recent warning and error-level log entries from the container, allowing you to trace incidents back to specific correlation IDs.

Why structured logging over free-text logs:

Structured logging with message templates produces JSON output where each property is a named field. This enables:

- Fast filtering and aggregation in Log Analytics (e.g., "show all errors for username X").
- Automatic type inference (numbers, booleans, GUIDs are not wrapped in quotes).
- Safe parameterization that prevents log injection attacks.
- Correlation across distributed components via the `X-Correlation-ID` header.

Free-text logs require regex parsing, are error-prone, and cannot be efficiently indexed or queried.

2. Part 2 — REST API (DTOs, Swagger, API Key Middleware)

2.1. Implementation

DTO Pattern (`src/CloudSoft.Web/Dtos/`):

- `JobPostingDto` — input for POST/PUT (user-specified fields only).
- `JobPostingOutputDto` — output for GET (includes server-generated fields: `Id` , `CreatedAt` , `UpdatedAt`).
- Separation between API boundary and internal entities: clients can never see or modify internal details.

ApiJobPostingsController

(`src/CloudSoft.Web/Controllers/ApiJobPostingsController.cs`):

- `[ApiController]` attribute enables automatic validation, model binding, and error handling.
- Full CRUD operations: `GET /api/JobPostings` , `GET /api/JobPostings/{id}` , `POST /api/JobPostings` , `PUT /api/JobPostings/{id}` , `DELETE /api/JobPostings/{id}` .
- `PATCH` operations for publish/close via dedicated endpoints.
- Returns semantic HTTP status codes: 200, 201, 204, 400, 404.

Why JobPostings? This resource was chosen because it is the core domain entity of the recruitment portal. Exposing it via REST enables external systems (e.g., job boards, internal tools) to query and manage postings programmatically, while MVC views continue to serve the browser-based admin interface.

DTO vs. Domain Entity separation:

```
// Domain entity – never exposed to API consumers
public class JobPosting : ICosmosEntity
{
    public string Id { get; set; }
    public string PartitionKey { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public JobPostingStatus Status { get; set; }
    // ... internal fields
}

// API output DTO – only what the client needs
public record JobPostingOutputDto(
    string Id,
    string Title,
    string Description,
    string Status,
    DateTime CreatedAt,
    DateTime UpdatedAt);
```

This separation prevents accidental data leakage, makes the API contract stable across entity changes, and allows different DTOs for different consumer needs.

Swagger / OpenAPI Documentation:

- Swagger UI is available at `/swagger` (always enabled, including production, for API documentation).
- `ApiKeyMiddleware` skips Swagger routes so the UI can be tested without an API key.
- A consumer finds the API by navigating to `https://<app-url>/swagger` or by fetching the OpenAPI spec at `https://<app-url>/swagger/v1/swagger.json`.

ApiKeyMiddleware (`src/CloudSoft.Web/Middleware/ApiKeyMiddleware.cs`):

- Validates the `x-API-Key` header against configured keys (`ApiKey:Keys` in app settings).
- Returns 401 (Unauthorized) if the header is missing, 403 (Forbidden) if the key is invalid.
- Protects all `/api/*` routes. MVC routes are unaffected.

How the key is kept out of version control:

The API key is stored in `appsettings.Development.json` (which is `.gitignore` d) for local development and in GitHub/Azure secrets for production. It is never committed to the repository. A client sends it via the `x-API-Key` HTTP header:

```
curl -H "X-API-Key: <your-key>" https://<app-url>/api/JobPostings
```

3. Part 3 — File Upload and Health Probes

3.1. File Upload (Azure Blob Storage)

IBlobService (`src/CloudSoft.Domain/IBlobService.cs`):

- Interface with two methods: `UploadAsync` and `IsAvailableAsync`.
- Enables dependency injection and testability.

AzureBlobService (`src/CloudSoft.Data/AzureBlobService.cs`):

- Uses the `Azure.Storage.Blobs` SDK.
- Authentication via Managed Identity: `BlobServiceClient` is constructed with only the account URL — no connection string, no storage key.
- Falls back to connection string if configured (for local development).

NoOpBlobService (`src/CloudSoft.Data/NoOpBlobService.cs`):

- No-operation implementation for local development without Azure Storage.
- All methods return immediately without doing anything.

ResumeUploadController (`src/CloudSoft.Web/Controllers/ResumeUploadController.cs`):

- PDF validation: checks MIME type (`application/pdf`) and file signature (`%PDF-` magic bytes).
- Max file size: 10 MB.
- Unique file name: `{Guid}_{originalFileName}` to avoid collisions.
- Synchronous upload (within 5-second timeout) — acceptable for files under 10 MB.

Conditional DI (`Program.cs`):

- `AzureBlobService` is registered only if `BlobStorage` configuration exists.
- Otherwise, `NoOpBlobService` is used automatically.

What is uploaded, by whom, and where:

Candidates (anonymous users) can upload PDF resumes through the `/ResumeUpload` page. Each file is stored in the `resumes` blob container within the Azure Storage Account. The container has `publicAccess: None`, so files are only accessible through the application's Managed Identity.

How Managed Identity authenticates to Blob Storage:

The application constructs a `BlobServiceClient` using only the account URL:

```
var blobServiceClient = new BlobServiceClient(new Uri(accountUrl));  
var containerClient = blobServiceClient.GetBlobContainerClient("resumes");
```

No credentials are passed. The `BlobServiceClient` automatically uses `DefaultAzureCredential`, which picks up the Container App's system-assigned Managed Identity at runtime. The Bicep template grants this identity the `Storage Blob Data Contributor` role on the storage account.

3.2. Health Probes (Deep Probes)

Three endpoints (via `MapHealthChecks` in `Program.cs`):

Endpoint	Purpose	Checks
<code>/health/live</code>	Liveness probe — is the process alive?	None (returns 200 if the process responds)
<code>/health/ready</code>	Readiness probe — is startup complete?	Tags <code>"ready"</code> (CosmosDB + Blob Storage)
<code>/health</code>	Detailed diagnostics	All checks with JSON response

CosmosHealthCheck (`src/CloudSoft.Web/HealthChecks/CosmosHealthCheck.cs`):

- Performs `ReadContainerAsync` to verify actual connectivity.
- 5-second timeout via `CancellationTokenSource`.
- Returns `Degraded` on timeout, `Unhealthy` on error.

BlobHealthCheck (`src/CloudSoft.Web/HealthChecks/BlobHealthCheck.cs`):

- Uses `IBlobService.IsAvailableAsync` to verify Blob Storage connectivity.
- 5-second timeout via `CancellationTokenSource`.
- Returns `Degraded` on timeout, `Unhealthy` on error.

How the readiness probe controls traffic:

The `/health/ready` endpoint is designed to be used as the Container App's readiness probe. When CosmosDB or Blob Storage is unavailable, the probe returns a non-200 status code. Container Apps then removes the instance from the load balancer pool, stopping traffic to that

replica until dependencies recover. This prevents requests from hitting an instance that cannot serve them properly.

Bugfix: Recursive `OperationCanceledException`

Both health checks had a critical bug where `catch (OperationCanceledException)` called `CheckHealthAsync` recursively, leading to infinite recursion and stack overflow. Fixed by returning `HealthCheckResult.Degraded()` on timeout instead.

4. Part 4 — Architecture Review and Reflection

4.1. Infrastructure Additions (`infra/main.bicep`)

Blob Storage provisioning:

- Storage Account (`Standard_LRS` , `StorageV2`) with `allowSharedKeyAccess: false` (requires Managed Identity).
- Blob container `resumes` with `publicAccess: None` (no anonymous access).

Managed Identity roles:

- Container App receives a `SystemAssigned` managed identity.
- `CosmosDB Built-in Data Contributor` role on the CosmosDB account.
- `Storage Blob Data Contributor` role on the Storage Account.
- Role assignments use implicit dependencies via `containerApp.identity.principalId` .

CosmosDB authentication migration:

- `disableLocalAuth: true` on CosmosDB (requires Managed Identity or RBAC).
- Bicep passes `CosmosDb__Endpoint` (account URI) instead of `ConnectionStrings__CosmosDb` .
- Application code (`CosmosExtensions.cs`) uses `DefaultAzureCredential` in production and connection string only in development.

Container App configuration:

- `BlobStorage__AccountUrl` environment variable with Blob Storage endpoint.
- `CosmosDb__Endpoint` , `CosmosDb__DatabaseName` , `CosmosDb__ContainerName` as environment variables.
- `ASPNETCORE_ENVIRONMENT` set to `Production` .
- No secrets array — all authentication is identity-based.

Security improvements:

- `disableLocalAuth: true` on CosmosDB (no account keys).
- `transport: 'auto'` on Container Apps ingress (enforces TLS).
- Storage Account with `allowSharedKeyAccess: false` (requires Managed Identity).
- API Key middleware for REST API protection.

4.2. CI/CD Pipeline (`.github/workflows/ci-cd.yml`)

The CI/CD pipeline is fully automated and handles the complete deployment lifecycle:

- **OIDC federation:** Azure login uses `azure/login@v2` with OIDC token (no `AZURE_CREDENTIALS` secret).
- **Bicep deployment:** Deploys all infrastructure resources including CosmosDB, Container Apps, and Blob Storage.
- **Managed Identity role assignments:** The pipeline automatically assigns `CosmosDB Built-in Data Contributor` and `Storage Blob Data Contributor` roles to the Container App's Managed Identity after Bicep deployment. A 30-second wait allows role propagation before the smoke test.
- **Smoke test with retry loop:** Uses `/health/live` with a retry loop (40 attempts × 15s, 10s per-request timeout) to handle Container Apps cold starts that can take 2-5 minutes on first deployment.
- **Parameterized:** `RESOURCE_GROUP` and `LOCATION` as environment variables.
- **Docker Hub:** Images tagged with commit SHA and `latest`.

4.3. Architecture Diagram

TODO

4.4. End-to-End Flow

A request enters the Container App through the ingress controller (TLS-terminated, `transport: 'auto'`). It passes through the middleware pipeline in order:

1. **CorrelationIdMiddleware** — assigns or reuses `X-Correlation-ID`, adds it to the logging scope.
2. **ApiKeyMiddleware** — validates `X-API-Key` header for `/api/*` routes (skips Swagger).
3. **Routing** — dispatches to the appropriate controller.
4. **Authentication / Authorization** — validates Identity cookies for MVC, role-based access for admin routes.

For a REST API request (`GET /api/JobPostings`):

- `ApiJobPostingsController` receives the request, calls `IJobPostingService.GetAllAsync()`.
- `JobPostingService` calls `IRepository<JobPosting>.GetAllAsync()`.
- `CosmosRepository<JobPosting>` executes a query against CosmosDB using the injected `CosmosClient`.
- Results are mapped to `JobPostingOutputDto` and returned as JSON.

For a file upload (`POST /ResumeUpload`):

- `ResumeUploadController` validates the file (PDF type, magic bytes, size limit).
- Calls `IBlobService.UploadAsync()` which, in production, uses `AzureBlobService`.
- `AzureBlobService` uploads to the `resumes` container via Managed Identity (no credentials).
- Returns the upload result to the browser.

For a health check (`GET /health/ready`):

- ASP.NET Core's built-in health check system runs all checks tagged `"ready"`.
- `CosmosHealthCheck` calls `ReadContainerAsync` on CosmosDB.
- `BlobHealthCheck` calls `IBlobService.IsAvailableAsync`.
- If all checks pass, returns 200 OK. If any fail, returns 503 Service Unavailable.
- Container Apps uses this to decide whether to send traffic to this replica.

4.5. Design Patterns Used

Pattern	Where	Purpose
Repository	<code>CosmosRepository<T></code> implementing <code>IRepository<T></code>	Abstracts CosmosDB access behind a generic interface. Enables swapping implementations and testing.
Dependency Injection	<code>Program.cs</code> DI container; all controllers and services receive dependencies via constructor injection	Inversion of control. Enables testability, conditional registration (<code>AzureBlobService</code> vs <code>NoOpBlobService</code>), and lifecycle management.
DTO	<code>JobPostingDto</code> , <code>JobPostingOutputDto</code> in <code>src/CloudSoft.Web/Dtos/</code>	Separates API contract from domain model. Prevents accidental data leakage.
Middleware	<code>CorrelationIdMiddleware</code> , <code>ApiKeyMiddleware</code>	Cross-cutting concerns (tracing, authentication) applied to all requests. Ordered pipeline execution.
Managed Identity	<code>AzureBlobService</code> , <code>CosmosExtensions.cs</code> (production path)	Zero-secret authentication to Azure services. Eliminates credential storage and rotation.
Deep Health Probe	<code>CosmosHealthCheck</code> , <code>BlobHealthCheck</code> with timeouts	Probes actual dependency health rather than just process liveness. Enables traffic control via readiness probe.
Service Layer	<code>IJobPostingService</code> / <code>JobPostingService</code>	Business logic between controllers and data access. Keeps controllers thin.
Conditional Registration	<code>Program.cs</code> — <code>AzureBlobService</code> vs <code>NoOpBlobService</code>	Same codebase works in development (no Azure) and production (full Azure).

4.6. Reflection

This assignment extended the recruitment portal from "Inlämningsuppgift 1" (Assignment 1) with observability, a REST API, file upload, and deep health probes. The most significant architectural change was migrating from connection-string-based authentication to Managed

Identity for both CosmosDB and Blob Storage. This eliminated all secret management from the application: no connection strings, no storage keys, no credentials stored anywhere. The `CosmosExtensions.cs` dual-auth pattern (connection string for development, `DefaultAzureCredential` for production) keeps local development simple while enforcing best practices in production.

The health probe system follows the Kubernetes-inspired liveness/readiness separation: `/health/live` answers only "is the process alive?" with no external calls, while `/health/ready` verifies that all dependencies are reachable before accepting traffic. This prevents the Container App from routing requests to instances that cannot serve them.

The DTO pattern, while simple, provides a critical boundary: the API contract is now decoupled from the domain model. If the `JobPosting` entity changes internally, the API output remains stable as long as the DTO contract is preserved. This is the foundation for API versioning and backward compatibility.

5. Azure Provisioning Guide — From Scratch

This section documents the complete set of steps required to provision and deploy the CloudSoft Recruitment Portal to Azure after all resources have been deleted.

5.1. Prerequisites

- **Azure CLI** (`az`) installed and logged in (`az login`)
- **GitHub CLI** (`gh`) installed and authenticated (`gh auth login`)
- **Docker Hub account** with write access (user `claes1981`)
- **Azure subscription** with Owner or User Access Administrator role
- Repository: `Claes1981/moln_fordj_inlamningsuppgifter`

5.2. Provisioning Script (`infra/provision.sh`)

The provisioning process is automated via `infra/provision.sh` , which handles:

1. **Service Principal creation** — Creates an Azure AD application registration with OIDC support
2. **Federated credential** — Links the service principal to GitHub Actions for OIDC federation
3. **Resource group creation** — Creates `cloudsoft-rg` in `northeurope`
4. **Owner role assignment** — Assigns Owner role (includes `roleAssignments/write` needed for MI role assignments)

5. GitHub secrets configuration — Sets `AZURE_CLIENT_ID` , `AZURE_TENANT_ID` , `AZURE_SUBSCRIPTION_ID`

To provision from scratch:

```
# Run the provisioning script (requires az and gh CLI)
chmod +x infra/provision.sh
./infra/provision.sh
```

Required GitHub secrets (set automatically by the script):

Secret	Purpose
<code>AZURE_CLIENT_ID</code>	Service Principal application ID
<code>AZURE_TENANT_ID</code>	Azure AD tenant ID
<code>AZURE_SUBSCRIPTION_ID</code>	Azure subscription ID
<code>DOCKERHUB_USERNAME</code>	Docker Hub username
<code>DOCKERHUB_TOKEN</code>	Docker Hub personal access token (read/write)

5.3. CI/CD Pipeline

After provisioning, push a commit to `main` or trigger the workflow manually. The pipeline will:

1. **build-and-push** — Build the Docker image and push to Docker Hub (tagged with commit SHA and `latest`)
2. **provision** — Log in to Azure via OIDC, deploy Bicep infrastructure, assign MI roles, and run a smoke test

The smoke test uses a **retry loop** (40 attempts × 15s, 10s per-request timeout) to handle Container Apps cold starts that can take 2-5 minutes on first deployment.

5.4. Verify Deployment

After the pipeline completes:

```
# Check deployed resources
az resource list --resource-group cloudsoft-rg --output table

# Get the app URL
APP_URL=$(az deployment group show \
  --resource-group cloudsoft-rg \
  --name main \
  --query properties.outputs.appUrl.value \
  --output tsv)

# Test health endpoints
curl -s "https://$APP_URL/health/live" # Liveness – should return 200
curl -s "https://$APP_URL/health/ready" # Readiness – may take a moment
curl -s "https://$APP_URL/health" # Deep probe – checks CosmosDB + Blob
```

5.5. Troubleshooting

Error	Cause	Fix
No subscriptions found for ...	Federated credential missing or wrong subject	Re-run <code>infra/provision.sh</code>
Authorization failed	Service principal lacks Owner role	Verify role assignment in Azure Portal
Health check fails on CosmosDB	Role assignment not yet propagated	Wait 5-10 minutes; Azure role propagation is eventual
Docker Hub push fails	Invalid or expired token	Generate new token at Docker Hub → Account Settings → Security
Smoke test fails (504)	Container Apps cold start	The retry loop handles this automatically (up to 10 minutes)

5.6. Post-Deployment: Managed Identity Role Propagation

After Bicep deployment, the Container App's Managed Identity receives two role assignments:

- **CosmosDB Built-in Data Contributor** on the CosmosDB account
- **Storage Blob Data Contributor** on the Storage Account

These are assigned automatically by the CI/CD pipeline and may take 30 seconds to 5 minutes to propagate. The smoke test retry loop accounts for this delay.

5.7. Cleanup (Optional)

To remove all resources:

```
# Delete resource group (removes all resources within it)
az group delete --name cloudsoft-rg --yes --no-wait

# Remove federated credential
CLIENT_ID=$(gh secret get AZURE_CLIENT_ID)
az ad app federated-credential delete --id $CLIENT_ID --name github-actions-main

# Remove service principal
az ad sp delete --id $CLIENT_ID
```

6. Sources

- Course exercises: <https://cloud-dev-25.educ8.se/exercises/>
 - 3-deployment/10-logging-and-monitoring/1-structured-logging-ilogger/
 - 4-services-and-apis/1-rest-api-and-dtos/1-rest-controllers-and-dtos/
 - 4-services-and-apis/1-rest-api-and-dtos/3-api-key-middleware/
 - 6-storage-and-resilience/1-uploads-and-deep-probes/1-mvc-uploads-and-pdf-validation/
 - 6-storage-and-resilience/1-uploads-and-deep-probes/2-cosmos-and-blob-via-managed-identity/
 - 6-storage-and-resilience/1-uploads-and-deep-probes/3-deep-health-probes-and-cleanup/